

Roteador Inteligente de LLMs Gratuitas por Tipo de Tarefa

Data: 2026-08-17 Status: IMPLEMENTADO Arquivo: scripts/task_router.py

1. Problema

O `detectar_llms_gratuitas.py` mapeia provedores ativos mas **não decide qual usar**. O operador precisa: 1. Escolher manualmente o provedor certo para cada tarefa 2. Trocar de provedor quando a cota acaba 3. Configurar o modelo correto no harness

2. Solução: Roteador por Tipo de Tarefa

Criar `scripts/task_router.py` que: - **Detecta tipo de tarefa** (coding, reasoning, creative, chat, embedding, vision) por keywords do prompt - **Seleciona melhor provedor** para aquela tarefa (ordem de preferência por latência/qualidade) - **Fallback automático** quando o provedor primário não tem chave configurada - **Quota tracking** em `~/.task_router_quota.json` — provedor que estourou cota é temporariamente bloqueado - **Output shell** (`export ORCA_PROVIDER=...` `ORCA_MODEL=...`) para integração com qualquer harness

3. Mapeamento Tarefa → Provedor

Tarefa	Provedores (ordem)	Modelo Recomendado
coding	groq → cerebras → google → open-router → nvidia	llama-3.3-70b-versatile
reasoning	google → groq → nvidia → open-router → cerebras	gemini-1.5-pro
creative	openrouter → google → groq → siliconflow → nvidia	meta-llama/llama-3.3-70b-instruct:free
analysis	google → groq → cerebras → open-router → nvidia	gemini-1.5-pro
chat	groq → google → openrouter → huggingface → siliconflow	llama-3.3-70b-versatile
embedding	huggingface → siliconflow → cohere → nvidia → fireworks	BGE-M3 / embed-v4
vision	google → openrouter → nvidia → siliconflow → fireworks	gemini-1.5-pro

4. Uso

```
# Linha de comando
python scripts/task_router.py "debug this Python function"
# → export ORCA_PROVIDER=groq ORCA_MODEL=llama-3.3-70b-versatile

# Integrado ao shell
eval $(python scripts/task_router.py "escreva um poema")
```

```
# Define ORCA_PROVIDER + ORCA_MODEL automaticamente

# Comando Orca em seguida
orca "continue a implementação"
# Usa groq + llama-3.3-70b-versatile
```

5. Arquivos

Arquivo	Descrição
scripts/task_router.py	Script principal — roteador + quota tracker
~/.task_router_quota.json	Estado de quotas por provedor (auto-gerenciado)

6. Integração com Orca/MiMoCode

O MiMoCode já tem ai-router, orcarouter, openrouter, unorouter, fastrouter, trustedrouter no cache de modelos. O task_router.py seta ORCA_PROVIDER e ORCA_MODEL que o Orca respeita quando definidos.

7. Quota Tracking

O script mantém ~/.task_router_quota.json com:

```
{
  "groq": {"remaining": 14400, "reset": "2026-08-18T00:00:00Z"},
  "cerebras": {"remaining": 1000000, "reset": "2026-08-18T00:00:00Z"}
}
```

Quando remaining chega a 0, o provedor é bloqueado até reset.

8. Verificação

1. python scripts/task_router.py "debug code" → groq + llama-3.3-70b-versatile
2. python scripts/task_router.py "write a poem" → openrouter + llama-3.3-70b:free
3. python scripts/task_router.py "analyze data" → google + gemini-1.5-pro
4. python scripts/task_router.py "embed search" → huggingface + BGE-M3
5. python scripts/task_router.py "describe image" → google + gemini-1.5-pro

Decisão: Boring/safe — usa .env + python-dotenv (já funcionando), não inventa middleware nem dependência externa. Roteamento por keyword + fallback em cascade. Quota tracking em JSON local (sem banco).